Temporizador/Contador

(Timer/Counter)

Microcontrolador PIC16F877A

MICROCHIP

PIC16F87XA

28/40/44-Pin Enhanced Flash Microcontrollers

Contando o Tempo com PIC

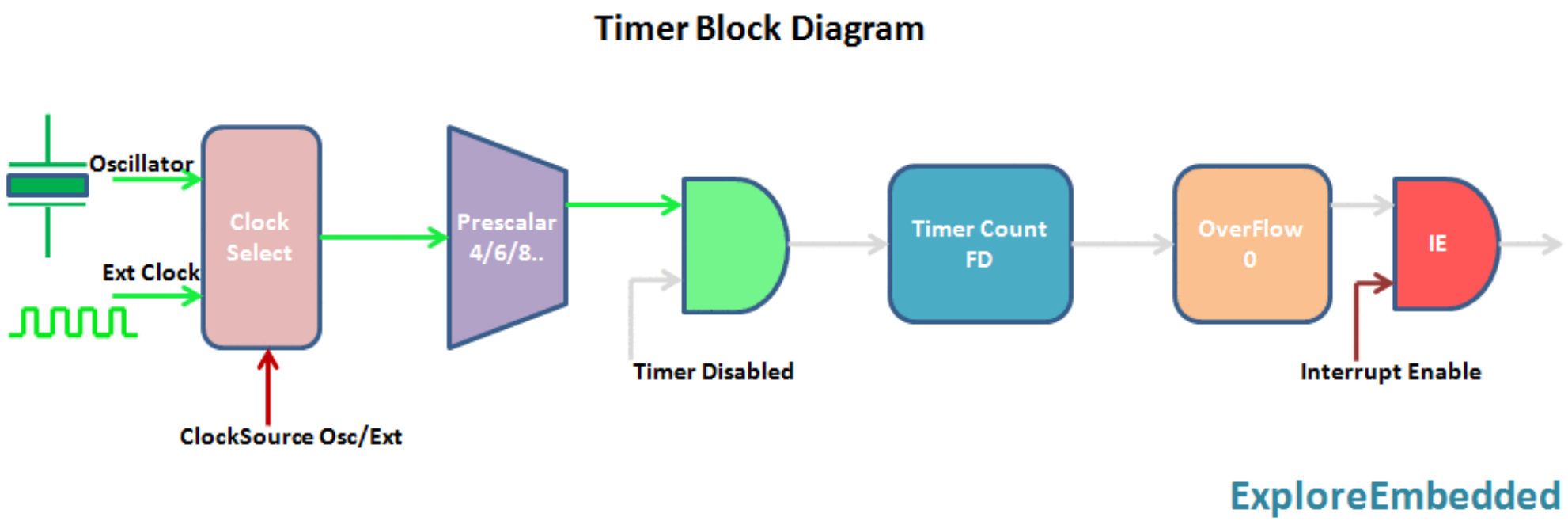
- Existem 3 maneiras de contarmos tempo com os microcontroladores PIC:
  - Contando **ciclo de máquina** por meio de **loopings**;
  - Contando os **ciclos de máquina** por meio dos **timers**;
  - Contando **pulsos externos** por meio dos **timers**.

Básicos dos Temporizadores

- Como o nome sugere, eles são usados para medir o **tempo** ou gerar o atraso preciso. O microcontrolador também pode gerar/medir os atrasos necessários **executando loops**, mas o temporizador/contador alivia a CPU dessa tarefa redundante e repetitiva, permitindo-lhe alocar o tempo máximo de processamento para outras tarefas.

- O **temporizador** nada mais é do que um simples **contador binário** que pode ser configurado para **contar pulsos de clock** (Interno/Externo). Assim que atingir o valor Max, ele voltará a zero configurando um sinalizador **OverFlow** e gerará a interrupção se habilitado.

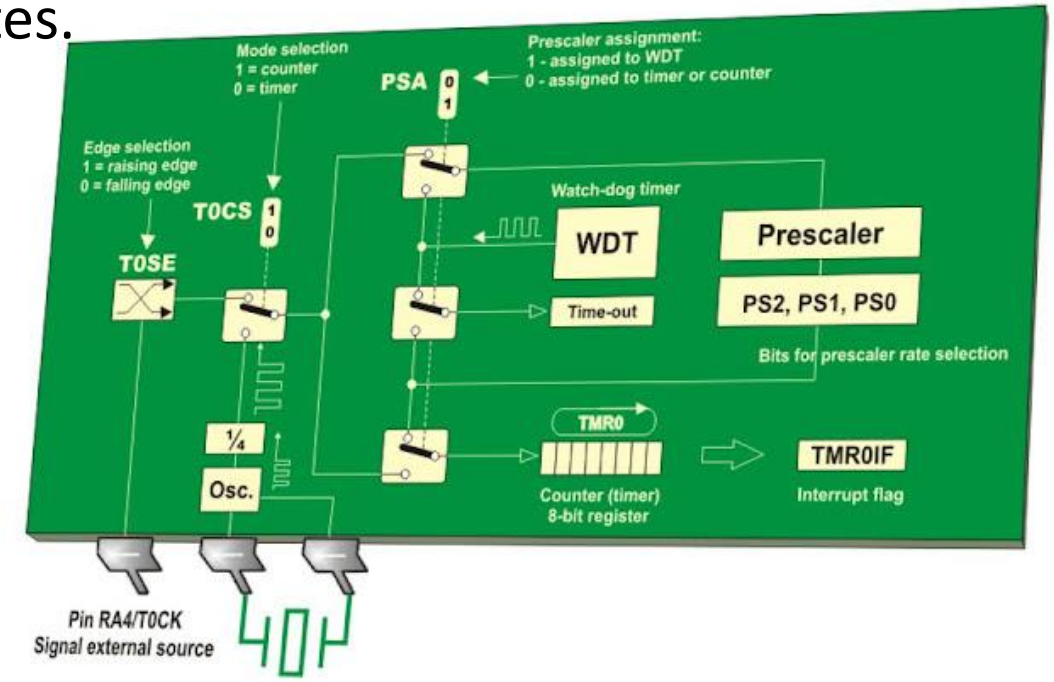
Diagrama de Blocos do Timer/Counter



Periféricos Timers

Descrição: **Timers** (temporizadores) são periféricos que:

- podem trabalhar como relógio ou contadores;
- independe da execução das instruções do PIC;
- estouro de contagem informado pelo PIC;
- PIC16F877A possui 3 timers independentes.



Módulos Timer do PIC

Timer	Size	Control Register	Count Register	Min Delay	Max Delay
TIMER0	8-bit	OPTION_REG	TMR0	0.2usec	13.107ms
TIMER1	16-bit	T1CON	TMR1H,TMR1L	0.2usec	104.857ms
TIMER2	8-bit	T2CON	TMR2	0.2usec	819usec

Timer0 do PIC16F877A

O **Temporizador/Contador TIMER0** possui as seguintes características:

- temporizador/contador de **8 bits**;
- leitura e gravação;
- software **prescaler** de 8 bits programável;
- Relógio **interno** ou **externo** selecionável;
- Interrupção no estouro (overflow) de FFh para 00h;
- Borda de seleção para relógio externo;

Registradores Associados ao TIMER0

TABLE 5-1: REGISTERS ASSOCIATED WITH TIMER0

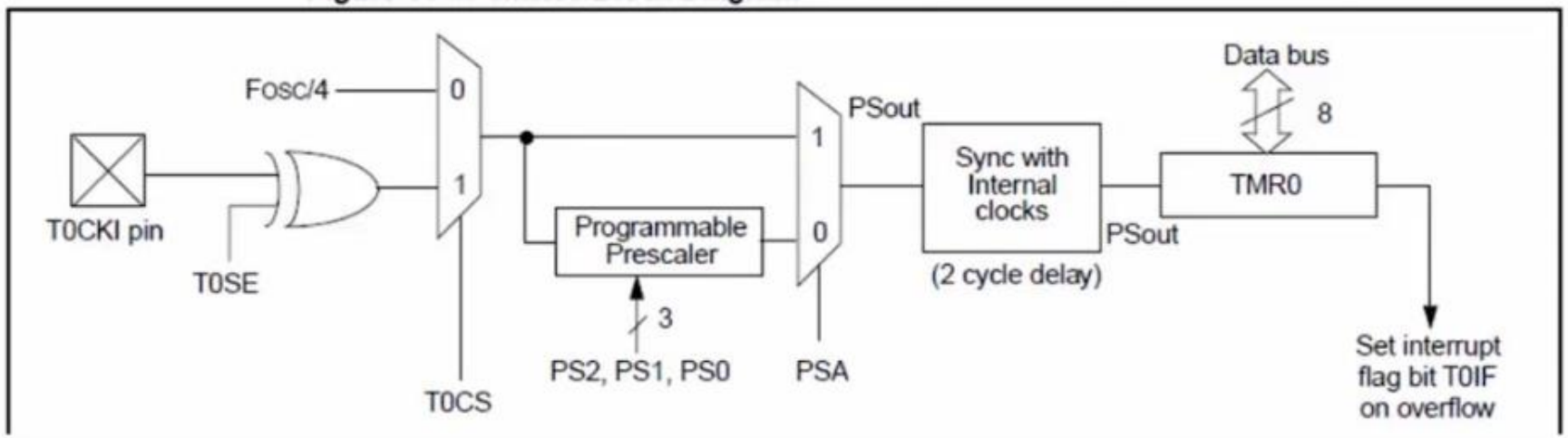
Address	Name	Bit 7	Bit 6	Bit 5	Bit 4	Bit 3	Bit 2	Bit 1	Bit 0	Value on: POR, BOR	Value on all other Resets
01h,101h	TMR0	Timer0 Module Register								xxxx xxxx	uuuu uuuu
0Bh,8Bh, 10Bh,18Bh	INTCON	GIE	PEIE	TMR0IE	INTE	RBIE	TMR0IF	INTF	RBIF	0000 000x	0000 000u
81h,181h	OPTION_REG	RBPU	INTEDG	T0CS	T0SE	PSA	PS2	PS1	PS0	1111 1111	1111 1111

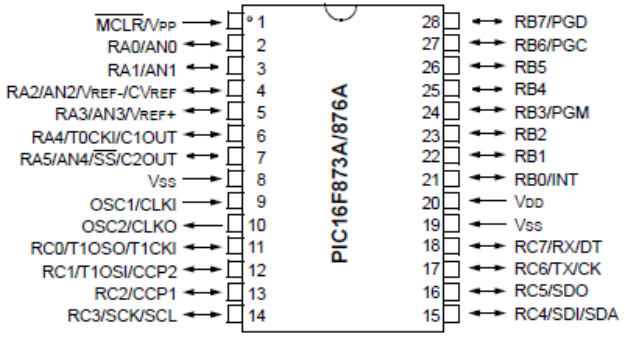
Legend: x = unknown, u = unchanged, - = unimplemented locations read as '0'. Shaded cells are not used by Timer0.

pág. 55 manual PIC16F877A

Diagrama interno do Timer0

Figure 11-1: Timer0 Block Diagram





Modo Contador

- O **TIMER0** funciona como contador, quando você usa como oscilador os **pulsos externos** aplicado no **pino RA4/T0CKIN**.
- Deve-se configurar o seguinte **registrador** (pág. 23 manual PIC16F877A):

OPTION_REG REGISTER (ADDRESS 81h, 181h)

R/W-1	R/W-1	R/W-1	R/W-1	R/W-1	R/W-1	R/W-1	R/W-1
$\overline{\text{RBPU}}$	INTEDG	T0CS	T0SE	PSA	PS2	PS1	PS0
bit 7							bit 0

T0CS - Define a fonte de clock do TIMER0:
1 - Externa (PINO RA4/T0CKIN) - Modo contador
0 - Interna (Cristal de quartzo) - Modo temporizador

PSA - Habilitação do prescaler (no modo CONTADOR é comum desabilitar)
1 - Desabilitado
0 - Habilitado

T0SE - Seleção da borda de transição do pino RA4/T0CKIN
1 - transição nivel alto-baixo (transição negativa)
0 - transição nivel baixo-alto (transição positiva)

PS2, PS1, PS0 - Seleção do prescaler(somente no modo temporizador);

bits	TMR0 rate
000	1:2
001	1:4
010	1:8
011	1:16
100	1:32
101	1:64
110	1:128
111	1:256

Prescaler é o divisor de frequência do clock do **TIMER0**. É usado no modo temporizador para poder obter tempos maiores.

Exemplo de Código de Configuração Contador

```
void main() {
        TRISB = 0;
        OPTION_REG = 0b10111000;     //modo contador
        TMR0 = 0;                    //Inicializa o registro TMR0 com o valor 0.

        while(1) {
          if (INTCON.TMR0IF) {  //  verifica se flag de overflow é 1.
             INTCON.TMR0IF = 0;    //limpa flag.
             TMR0 = 0;  // inicializa com 0 ou qualquer outro valor.
             portb = ~portb; //quando acontecer o overflow (a cada 256 pulsos) inverte o sinal da porta B.
          }
                delay_ms(10);
        }
        }
```

- O registrador **TMR0** armazena o estado inicial da contagem. Possui **8 bits**, por isso **conta de 0 a 255**.
- Quando acontece o **overflow** ele retorna a **contagem inicial a 0**.
- O bit de **overflow** é o **INTCON.TMR0IF** e deve ser **limpo** por software.

Modo Temporizador

- O temporizador do **TIMER0** funciona quando você usa o **oscilador interno** do PIC ou o **cristal de quartzo externo**.
- Deve-se configurar o seguinte **registrador** (pág. 23 manual Microchip):

OPTION_REG REGISTER (ADDRESS 81h, 181h)

R/W-1	R/W-1	R/W-1	R/W-1	R/W-1	R/W-1	R/W-1	R/W-1
$\overline{\text{RBPU}}$	INTEDG	T0CS	T0SE	PSA	PS2	PS1	PS0
bit 7							bit 0

T0CS - Define a fonte de clock do TIMER0:
1 - Externa (PINO RA4/T0CKIN) - Modo contador
0 - Interna (Cristal de quartzo) - Modo temporizador

PSA - Habilitação do prescaler (no modo CONTADOR é comum desabilitar)
1 - Desabilitado
0 - Habilitado

T0SE - Seleção da borda de transição do pino RA4/T0CKIN
1 - transição nivel alto-baixo (transição negativa)
0 - transição nivel baixo-alto (transição positiva)

PS2, PS1, PS0 - Seleção do prescaler(somente no modo temporizador);

bits	TMR0 rate
000	1:2
001	1:4
010	1:8
011	1:16
100	1:32
101	1:64
110	1:128
111	1:256

Prescaler é o divisor de frequência do clock do **TIMER0**. É usado no modo temporizador para poder obter tempos maiores.

Cálculo do Tempo de Overflow

- **Tout = ciclo de máquina * prescaler * (256 – TMR0);**
- ciclo de máquina = 4 / fosc;
- **fosc (fclk)** é a freqência do clock do cristal oscilador.

EXEMPLO:
O clock de um PIC é de 4MHz, utlizado o prescaler de 1:256 e contagem de 0 a 255, qual será o tempo?

ciclo de máquina = 4/4 = 1us.

Tout = 1 * 256 * 256;
Tout = 65536 us ou 65,536 ms

para ter tempos ainda maiores, como alguns segundos,um dos meios de se conseguir é utilizando a interrupção.

Exemplo de Código de Configuração
Temporizador

```
void main() {
        TRISB = 0;
        OPTION_REG = 0b10111000;     //modo contador
        TMR0 = 0;                    //Inicializa o registro TMR0 com o valor 0.

        while(1) {
          if (INTCON.TMR0IF) {  //  verifica se flag de overflow é 1.
             INTCON.TMR0IF = 0;    //limpa flag.
             TMR0 = 0;  // inicializa com 0 ou qualquer outro valor.
             portb = ~portb; //quando acontecer o overflow (a cada 256 pulsos) inverte o sinal da porta B.
          }
                delay_ms(10);
        }
        }
```

- O registrador **TMR0** armazena o estado inicial da contagem. Possui **8 bits**, por isso **conta de 0 a 255**.
- Quando acontece o **overflow** ele retorna a **contagem inicial a 0**.
- O bit de **overflow** é o **INTCON.TMR0IF** e deve ser **limpo** por software.

Cálculo dos Valores de Count, Fout e TMR0

- Usando o **clock interno**:

$$fout = \frac{fclk}{4 * \text{Prescaler} * (256 - \text{TMR0}) * \text{Count}} \quad where \quad Tout = \frac{1}{fout}$$

- **fout:** A frequência de saída após a divisão.
- **Tout**: O tempo do ciclo após a divisão.
- **4**: A divisão do **clock original** (fclk) por 4, quando se utiliza cristal interno como clock (e não oscilador externo).
- **Count**: Um valor numérico a ser colocado para obter a frequência de saída desejada - **fout**.
- **(256 - TMR0)**: O número de vezes no temporizador será contado com base no registro **TMR0**.

EXEMPLO: Usando clock interno

- Suponha que queiramos criar um **atraso** de **0,5 segundo** em nosso programa usando **Timer0**. Qual é o valor de **Contagem**?
- Primeiro, vamos supor que a divisão de frequência pelo **Prescaler** será **1:256**. Segundo, vamos definir **TMR0=0**.
- $\boldsymbol{fout = \frac{fclk}{4*Prescaler*(256-TMR0)*Count} = 2\ Hz\ (frequência\ necessária)}$

$$\boldsymbol{fout = \frac{1}{Tout} = \frac{1}{0,5} = 2\ Hz}$$

$$\boldsymbol{Count = \frac{4\ MHz}{4 * 256 * (256 - 0) * 2\ Hz} = 7,6294 \cong 8}$$

Cálculo dos Valores de Count,Fout e TMR0

- Usando o **clock externo**:

$$fout = \frac{fclk}{\text{Prescaler} * (256 - \text{TMR0}) * \text{Count}} \quad where \quad Tout = \frac{1}{fout}$$

- **fout**: A frequência de saída após a divisão.
- **Tout**: O tempo do ciclo após a divisão.
- **Count**: Um valor numérico a ser colocado para obter a frequência de saída desejada - **fout**.
- **(256 - TMR0)**: O número de vezes no temporizador será contado com base no registro **TMR0**.

EXEMPLO: Usando clock externo

- Qual é a frequência de saída - **Fout**, quando o oscilador externo é **100 kHz** e **Count=8**?
- Primeiro, vamos supor que a divisão de frequência pelo **Prescaler** será **1:256**. Segundo, vamos definir **TMR0=0**.
- $$\boldsymbol{fout = \frac{fclk}{Prescaler*(256-TMR0)*Count} = 2\ Hz\ (frequência\ necessária)}$$

$$\boldsymbol{fout = \frac{100\ KHz}{256*(256-0)*8} = 0,19\ Hz}$$

$$\boldsymbol{Tout = \frac{1}{fout} = \frac{1}{0,19} = 5,243\ s}$$